Федеральное государственное бюджетное образовательное учреждение высшего образования
«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Системы обработки больших данных»

на тему:
«Сверка результатов и повторная корректировка витрины Практическая работа 12»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверил
Преподаватель:
Гуцол Д.В.

Оценка:_____
Дата: 10.11.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

Цель работы

Цель — воспроизвести предварительный снимок витрины продаж по пятиминутным окнам с отставанием watermark на две минуты, сопоставить его с пакетным эталоном, исправить запоздавшую покупку E009 полной заменой строки по ключу и редакции, проверить идемпотентность повтора и защитные отказы, а затем выполнить семь проверок качества аналитической витрины (Практическая работа 12) и исправить рабочую копию fixed.csv.

1. Подготовка среды и исходные события

Работа выполнена локально на Windows с Python 3.14. Использован учебный набор из 12 событий data/arrivals.csv. В суммы входят только purchase/success; E004 — просмотр, E006 — неуспешная покупка. Окна: [10:00,10:05), [10:05,10:10), [10:10,10:15), [10:15,10:20).

```
py -3 lab.py check

ENV=OK
PYTHON=3.14.2
SOURCE_SHA256=PASS (96ebed13b8d2139d...)
EVENTS=12
READY
```

Рисунок 1. Проверка среды: SOURCE_SHA256=PASS, EVENTS=12

```
type data□rrivals.csv

event_id,event_time,arrival_time,event_type,status,amount
E001,10:01:00,10:01:30,purchase,success,100.00
E002,10:02:00,10:02:20,purchase,success,200.00
E003,10:05:00,10:05:10,purchase,success,300.00
E004,10:06:00,10:06:05,view,success,0.00
E005,10:03:00,10:06:30,purchase,success,400.00
E006,10:07:00,10:07:10,purchase,fail,250.00
E007,10:08:00,10:08:15,purchase,success,500.00
E010,10:10:00,10:10:30,purchase,success,600.00
E008,10:11:00,10:11:20,purchase,success,800.00
E011,10:12:00,10:12:10,purchase,success,900.00
E009,10:07:00,10:13:00,purchase,success,700.00
E012,10:16:00,10:16:20,purchase,success,1000.00
```

Рисунок 2. Исходные события arrivals.csv (порядок = поступление)

2. Предварительный снимок и пакетный эталон

Ручные суммы успешных покупок: окно 10:00 — E001+E002+E005 = 700; 10:05 — E003+E007+E009 = 1500; 10:10 — E010+E008+E011 = 2300; 10:15 — E012 = 1000. Итого эталон 5500 (10 покупок).

В режиме watermark lag=2 мин до E009 max(event_time)=10:12, watermark_before=10:10. Конец окна E009 равен 10:10, условие window_end ≤ watermark_before выполняется, поэтому E009 получает решение LATE и не входит в preliminary. Снимок: 9 покупок, 4800 руб.

```
py -3 lab.py snapshot

SNAPSHOT (watermark lag = 2 minutes)
accepted_purchases=9
accepted_sum_rub=4800.00
late_events=1
LATE E009 amount=700.00 window=10:05
windows:
  10:00: count=3 sum=700.00 rev=1
  10:05: count=2 sum=800.00 rev=1
  10:10: count=3 sum=2300.00 rev=1
  10:15: count=1 sum=1000.00 rev=1
```

Рисунок 3. Snapshot: 4800 руб., LATE E009 на 700

```
py -3 lab.py reference

REFERENCE (batch, no late drop)
purchases=10
sum_rub=5500.00
windows:
  10:00: count=3 sum_cents=70000 (700.00 rub)
  10:05: count=3 sum_cents=150000 (1500.00 rub)
  10:10: count=3 sum_cents=230000 (2300.00 rub)
  10:15: count=1 sum_cents=100000 (1000.00 rub)
```

Рисунок 4. Reference: 10 покупок, 5500 руб.

3. Аудит расхождения

Сверка по ключам окон показала единственное расхождение в 10:05: before_count=2 / before_cents=80000 против reference_count=3 / reference_cents=150000; delta_count=1, delta_cents=70000. Журнал late.csv содержит E009. Остальные окна совпали. Контроль только общего итога недостаточен: ошибки +100 и −100 в разных окнах сохранят день, но исказят распределение.

```
py -3 lab.py audit

AUDIT preliminary vs reference
10:00: before=3/700.00 ref=3/700.00 delta_count=0 delta_cents=0
10:05: before=2/800.00 ref=3/1500.00 delta_count=1 delta_cents=70000
10:10: before=3/2300.00 ref=3/2300.00 delta_count=0 delta_cents=0
10:15: before=1/1000.00 ref=1/1000.00 delta_count=0 delta_cents=0
--- late.csv ---
event_id,event_time,arrival_time,amount,window_start,sum_cents,reason
E009,10:07:00,10:13:00,700.00,10:05,70000,window_end<=watermark_before
```

Рисунок 5. Audit: расхождение только в окне 10:05

```
type results\late.csv

event_id,event_time,arrival_time,amount,window_start,sum_cents,reason
E009,10:07:00,10:13:00,700.00,10:05,70000,window_end<=watermark_before
```

Рисунок 6. late.csv — недостающие 700 руб. (E009)

4. Неправильный повтор прибавления и patch

Прогноз: одно прибавление E009 к 4800 даст 5500, два — 6200. Факт совпал. Окно 10:05 меняется 800→1500→2200; после второго ADD покупок 11 вместо 10. Ошибка в способе обновления: повторно применяется одно и то же исправление.

```
py -3 lab.py naive

NAIVE repeated ADD of late events
once: purchases=10 sum_rub=5500.00
twice: purchases=11 sum_rub=6200.00
window 10:05 once=150000 twice=220000
```

Рисунок 7. Naive: once=5500, twice=6200

Корректировка передаёт полное значение строки: ключ window_start=10:05, count=3, sum_cents=150000, revision=2. Разница 700 — только дельта; замена окна на 700 потеряла бы ранее учтённые 800, а прибавление 1500 к 800 дало бы лишнюю сумму. Выбрана полная замена строки.

```
py -3 lab.py patch

PATCH full replacement values
key=10:05 count=3 sum_cents=150000 revision=2
```

Рисунок 8. Patch: полная замена значения окна 10:05

```
type results\patch.csv

window_start,count,sum_cents,revision
10:05,3,150000,2
```

Рисунок 9. patch.csv — sum_cents в копейках

5. Применение, повтор и защитные отказы

Первое применение: REPLACE, редакция 1→2, окно 10:05 = 3 покупки / 1500 руб., общий итог 5500. Состояние сохранено в state.json.

```
py -3 lab.py apply

STATE_ALREADY_CORRECTED -> showing saved 06_apply.txt
APPLY first correction
10:05: action=REPLACE rev 2 count=3 sum_cents=150000
total_purchases=10
total_sum_rub=5500.00
```

Рисунок 10. Apply: REPLACE и итог 5500

Повтор той же корректировки на сохранённом состоянии: NOOP_SAME. after_first и after_repeat совпали. Это проверка идемпотентности на сохранённой витрине, а не два независимых расчёта с нуля.

```
py -3 lab.py repeat

REPEAT same patch on saved state
10:05: action=NOOP_SAME
total_purchases=10
total_sum_rub=5500.00
```

Рисунок 11. Repeat: NOOP_SAME

Guards на копии состояния: REJECT_STALE (редакция 1 после 2) и REJECT_CONFLICT (редакция 2 с суммой 1600). Оба отказа сохраняют 5500. Точный повтор даёт NOOP_SAME.

```
py -3 lab.py guards

GUARDS on copy of corrected state
stale: action=REJECT_STALE total_rub=5500.00 purchases=10
conflict: action=REJECT_CONFLICT total_rub=5500.00 purchases=10
same: action=NOOP_SAME total_rub=5500.00 purchases=10
```

Рисунок 12. Guards: REJECT_STALE и REJECT_CONFLICT

```
py -3 lab.py compare

COMPARE
PASS window 10:00 count/amount match reference
PASS window 10:05 count/amount match reference
PASS window 10:10 count/amount match reference
PASS window 10:15 count/amount match reference
preliminary_total=4800.00 (9)
reference_total=5500.00 (10)
corrected_total=5500.00 (10)
naive_twice_total=6200.00 (11)
RESULT=PASS
```

Рисунок 13. Compare: исправленная витрина = эталон; naive twice = 6200

6. Практическая работа 12 — семь проверок качества

Эталонные суммы в копейках: 70000, 150000, 230000, 100000; количества 3,3,3,1; редакции 1,2,1,1. Строк витрины четыре (по окнам), а не десять (по событиям).

PUBLISH=ALLOW только при семи PASS: SCHEMA, UNIQUE, COVERAGE, TOTALS, VALUES, REVISION, SOURCE. SKIP не считается успехом.

```
py -3 lab.py audit clean

AUDIT case=clean
PUBLISH=ALLOW
SCHEMA=PASS
UNIQUE=PASS
COVERAGE=PASS
TOTALS=PASS
VALUES=PASS
REVISION=PASS
SOURCE=PASS
```

Рисунок 14. clean — все PASS, PUBLISH=ALLOW

Прогноз для missing (без окна 10:15): 9 покупок, 4500 руб., BLOCK. Факт: COVERAGE=FAIL (MISSING 10:15), TOTALS=FAIL, VALUES/REVISION=SKIP.

```
py -3 lab.py audit missing

AUDIT case=missing
PUBLISH=BLOCK
SCHEMA=PASS
UNIQUE=PASS
COVERAGE=FAIL
TOTALS=FAIL
VALUES=SKIP
REVISION=SKIP
SOURCE=PASS
detail: MISSING 10:15
detail: TOTALS got count=9 cents=450000
detail: VALUES/REVISION skipped due to coverage
```

Рисунок 15. missing — BLOCK из-за отсутствия 10:15

В duplicate повторён ключ 10:05. UNIQUE=FAIL при COVERAGE=PASS: все нужные ключи есть, но один встречается дважды. VALUES/REVISION получают SKIP — программа не выбирает произвольно строку.

```
py -3 lab.py audit duplicate

AUDIT case=duplicate
PUBLISH=BLOCK
SCHEMA=PASS
UNIQUE=FAIL
COVERAGE=PASS
TOTALS=SKIP
VALUES=SKIP
REVISION=SKIP
SOURCE=PASS
detail: DUPLICATE_KEY
detail: VALUES/REVISION/TOTALS skipped due to duplicate key
```

Рисунок 16. duplicate — UNIQUE=FAIL, COVERAGE=PASS

В balanced ошибки +10000 и −10000 копеек в окнах 10:00 и 10:05 компенсировались: TOTALS=PASS, VALUES=FAIL. Итоговый оборот дня верный, распределение по времени — нет.

```
py -3 lab.py audit balanced

AUDIT case=balanced
PUBLISH=BLOCK
SCHEMA=PASS
UNIQUE=PASS
COVERAGE=PASS
TOTALS=PASS
VALUES=FAIL
REVISION=PASS
SOURCE=PASS
detail: VALUES 10:00: got 3/80000 expected 3/70000
detail: VALUES 10:05: got 3/140000 expected 3/150000
```

Рисунок 17. balanced — TOTALS=PASS при VALUES=FAIL

stale: числа верны, но revision 10:05=1 вместо 2 → REVISION=FAIL. foreign: CSV верный, но source_sha256 не совпадает с манифестом → SOURCE=FAIL. Совпадения чисел недостаточно без редакции и связи с источником.

```
py -3 lab.py audit stale

AUDIT case=stale
PUBLISH=BLOCK
SCHEMA=PASS
UNIQUE=PASS
COVERAGE=PASS
TOTALS=PASS
VALUES=PASS
REVISION=FAIL
SOURCE=PASS
detail: REVISION 10:05: got 1 expected 2
```

Рисунок 18. stale — REVISION=FAIL

```
py -3 lab.py audit foreign

AUDIT case=foreign
PUBLISH=BLOCK
SCHEMA=PASS
UNIQUE=PASS
COVERAGE=PASS
TOTALS=PASS
VALUES=PASS
REVISION=PASS
SOURCE=FAIL
detail: SOURCE_SHA mismatch
```

Рисунок 19. foreign — SOURCE=FAIL

invalid: в count окна 10:05 записано three. SCHEMA=FAIL (INVALID_ROW line=3); TOTALS/VALUES/REVISION=SKIP. Замена three на 0 была бы ложным исправлением — ожидается count=3. Обработчик должен зафиксировать ошибку и заблокировать публикацию.

```
py -3 lab.py audit invalid

AUDIT case=invalid
PUBLISH=BLOCK
SCHEMA=FAIL
UNIQUE=PASS
COVERAGE=PASS
TOTALS=SKIP
VALUES=SKIP
REVISION=SKIP
SOURCE=PASS
detail: INVALID_ROW line=3 types count=three
```

Рисунок 20. invalid — SCHEMA=FAIL

7. Исправление fixed.csv и итоговый протокол

В рабочей копии исправлены ровно два значения sum_cents: 10:00: 80000→70000 (100+200+400); 10:05: 140000→150000 (300+500+700). count, window_start и revision не менялись. После audit fixed — семь PASS, PUBLISH=ALLOW; повторный запуск без изменения файла дал тот же результат.

```
type cases□ixed.csv

window_start,count,sum_cents,revision
10:00,3,70000,1
10:05,3,150000,2
10:10,3,230000,1
10:15,1,100000,1
```

Рисунок 21. Исправленный cases/fixed.csv

```
py -3 lab.py audit fixed

AUDIT case=fixed
PUBLISH=ALLOW
SCHEMA=PASS
UNIQUE=PASS
COVERAGE=PASS
TOTALS=PASS
VALUES=PASS
REVISION=PASS
SOURCE=PASS
```

Рисунок 22. audit fixed — PUBLISH=ALLOW

```
py -3 lab.py summary

SUMMARY  SC UN CV TO VA RE SO  PUBLISH
clean      P P P P P P P  ALLOW
missing    P P F F - - P  BLOCK
duplicate  P F P - - - P  BLOCK
balanced   P P P P F P P  BLOCK
stale      P P P P P F P  BLOCK
invalid    F P P - - - P  BLOCK
foreign    P P P P P P F  BLOCK
fixed      P P P P P P P  ALLOW
```

Рисунок 23. Сводный протокол по всем примерам

8. Выводы и границы модели

1) E009 не входит в предварительные 4800 из-за правила watermark, но входит в пакетные 5500, где поздние записи не отбрасываются.

2) Разница 700 — дельта к старому результату; полное значение окна — 1500. Корректное обновление — REPLACE строки по ключу и новой редакции.

3) Повтор проверяется на сохранённом state.json: NOOP_SAME подтверждает идемпотентность операции обновления в данном локальном опыте.

4) REJECT_STALE отклоняет меньшую редакцию; REJECT_CONFLICT — ту же редакцию с иным содержимым. Оба случая сохраняют уже принятый результат.

5) Общей суммы 5500 недостаточно для публикации: нужны уникальность ключей, покрытие окон, значения каждой строки, редакции и source_sha256. Опыт не доказывает exactly-once распределённой системы: нет конкурирующих узлов, аварии посреди сохранения и транзакционной связи с источником сообщений. Лишний ключ 10:20 обнаружила бы проверка COVERAGE.

Контрольные ответы

Ч.1 Q1: E009 опоздала относительно watermark (window_end≤watermark_before), поэтому не в 4800; пакетный эталон учитывает все purchase/success, включая E009.

Ч.1 Q2: 700 — недостающая дельта; 1500 — полная сумма окна после включения E009.

Ч.1 Q3: иначе это два независимых расчёта с нуля, а не повтор операции над состоянием.

Ч.1 Q4: STALE — устаревший номер редакции; CONFLICT — тот же номер при другом содержимом.

Ч.1 Q5: локальный последовательный сценарий не моделирует распределённые гонки и сбои.

Ч.2: 5500 / четыре строки / один хеш сами по себе не доказывают полноту реального потока; нужен полный набор из семи проверок на согласованном снимке известных 12 событий.